

DEVSPHERE

INTERNSHIP PROGRAM

WEEK 4 TASK REPORT

Artificial Intelligence (AI) & Machine Learning (ML)

Neural Networks Basics & Classification Models

Intern	Saud Masood
Domain	Artificial Intelligence & Machine Learning
Internship	AI/ML Internship
Week	04
AI Task	Neural Networks Basics
ML Task	Classification Models
Report Date	23 September 2026

1. Week 4 Overview

Week 4 contains two practical tasks: an Artificial Intelligence task on Neural Networks Basics and a Machine Learning task on Classification Models. The report presents the task requirements, implementation, code, screenshot spaces, output areas, workflow explanations, and conclusions for both tasks.

2. Artificial Intelligence (AI) — Week 4 Task

Topic: Neural Networks Basics

The AI Week 4 task focuses on implementing a simple neural network. The objective is to create a basic neural network model, train it on a dataset, make predictions, show the results, and briefly explain the workflow.

AI Task Requirements

- Create a basic neural network model.
- Train the model on a dataset.
- Show predictions/results.
- Explain the workflow briefly.
- Output: Basic neural network implementation.

AI Dataset

The student_scores_week4.csv dataset contains StudyHours and ExamScore. StudyHours is used as the input feature, while ExamScore is the value the neural network learns to predict.

```
StudyHours,ExamScore
1,50
2,55
3,60
4,65
5,70
6,75
7,80
8,85
9,90
10,95
```

AI Task Implementation

For this task, I used Pandas to load the dataset and scikit-learn's MLPRegressor to create a simple neural network. The input data is scaled before training. The network contains one hidden layer with 10 neurons and uses the ReLU activation function. After training, the model predicts exam scores for new study-hour values.

AI Python Code

```
import pandas as pd
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

print("=====")
print("  AI WEEK 4 - NEURAL NETWORK")
print("=====")

# Load dataset
```

```

df = pd.read_csv("student_scores_week4.csv")

print("\n===== DATASET =====")
print(df)

# Input feature and target
X = df[["StudyHours"]]
y = df["ExamScore"]

# Scale input data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Create Neural Network model
model = MLPRegressor(
    hidden_layer_sizes=(10,),
    activation="relu",
    solver="lbfgs",
    max_iter=5000,
    random_state=42
)

# Train model
model.fit(X_scaled, y)

print("\n===== MODEL TRAINING =====")
print("Neural Network trained successfully!")

# New data for prediction
test_data = pd.DataFrame({
    "StudyHours": [4, 6, 8]
})

# Scale test data
test_scaled = scaler.transform(test_data)

# Make predictions
predictions = model.predict(test_scaled)

print("\n===== PREDICTIONS =====")

for hours, score in zip(test_data["StudyHours"], predictions):
    print(
        f"Study Hours: {hours} -> "
        f"Predicted Exam Score: {score:.2f}"
    )

print("\n===== NETWORK INFORMATION =====")
print("Input Layer: 1 feature")
print("Hidden Layer: 10 neurons")
print("Activation Function: ReLU")
print("Output: Predicted Exam Score")

print("\n=====")
print("    BASIC NEURAL NETWORK COMPLETED")
print("=====")

```

AI Code Screenshot

```

1 import pandas as pd
2 from sklearn.neural_network import MLPRegressor
3 from sklearn.preprocessing import StandardScaler
4
5 print("=====")
6 print("  AI WEEK 4 - NEURAL NETWORK")
7 print("=====")
8
9 # Load dataset
10 df = pd.read_csv("student_scores.csv")
11
12 print("\n===== DATASET =====")
13 print(df)
14
15 # Input feature and target
16 X = df[["StudyHours"]]
17 y = df["ExamScore"]
18
19 # Scale input data
20 scaler = StandardScaler()
21 X_scaled = scaler.fit_transform(X)
22
23 # Create Neural Network model
24 model = MLPRegressor(
25     hidden_layer_sizes=(10,),
26     activation="relu",
27     solver="lbfgs",

```

```

21 X_scaled = scaler.fit_transform(X)
22
23 # Create Neural Network model
24 model = MLPRegressor(
25     hidden_layer_sizes=(10,),
26     activation="relu",
27     solver="lbfgs",
28     max_iter=5000,
29     random_state=42
30 )
31
32 # Train model
33 model.fit(X_scaled, y)
34
35 print("\n===== MODEL TRAINING =====")
36 print("Neural Network trained successfully!")
37
38 # New data for prediction
39 test_data = pd.DataFrame({
40     "StudyHours": [4, 6, 8]
41 })
42
43 # Scale test data
44 test_scaled = scaler.transform(test_data)
45
46 # Make predictions
47 predictions = model.predict(test_scaled)
48
49 print("\n===== PREDICTIONS =====")

```

```
49 print("\n===== PREDICTIONS =====")
50
51 for hours, score in zip(test_data["StudyHours"], predictions):
52     print(
53         f"Study Hours: {hours} -> "
54         f"Predicted Exam Score: {score:.2f}"
55     )
56
57 print("\n===== NETWORK INFORMATION =====")
58 print("Input Layer: 1 feature")
59 print("Hidden Layer: 10 neurons")
60 print("Activation Function: ReLU")
61 print("Output: Predicted Exam Score")
62
63 print("\n=====")
64 print("    BASIC NEURAL NETWORK COMPLETED")
65 print("=====")
```

AI Week 4 — Neural Network Code Screenshot

AI Output Screenshot

```

***  =====
      AI WEEK 4 - NEURAL NETWORK
      =====

      ===== DATASET =====
            StudyHours  ExamScore
0              1         50
1              2         55
2              3         60
3              4         65
4              5         70
5              6         75
6              7         80
7              8         85
8              9         90
9             10         95

      ===== MODEL TRAINING =====
      Neural Network trained successfully!

      ===== PREDICTIONS =====
      Study Hours: 4 -> Predicted Exam Score: 65.00
      Study Hours: 6 -> Predicted Exam Score: 75.00
      Study Hours: 8 -> Predicted Exam Score: 85.00

      ===== NETWORK INFORMATION =====
      Input Layer: 1 feature
      Hidden Layer: 10 neurons
      Activation Function: ReLU
      Output: Predicted Exam Score

```

AI Week 4 — Neural Network Output Screenshot

AI Workflow Explanation

- Load the student score dataset using Pandas.
- Select StudyHours as the input and ExamScore as the target.
- Scale the input values before training.
- Create a neural network with one hidden layer containing 10 neurons.
- Train the neural network on the dataset.
- Provide new study-hour values to the trained model.
- Generate and display the predicted exam scores.

AI Result

The expected result is a successfully trained basic neural network that accepts study hours as input and produces predicted exam scores. The Google Colab output screenshot should be inserted above to document the actual execution.

3. Machine Learning (ML) — Week 4 Task

Topic: Classification Models

The ML Week 4 task focuses on implementing a classification model. The objective is to use a classification dataset, train a classification model, predict outputs, and show the model's accuracy score.

ML Task Requirements

- Use a classification dataset.
- Train a classification model.
- Predict outputs.
- Show accuracy score.
- Output: Working classification model.

ML Dataset

The student_classification.csv dataset contains StudyHours, Attendance, and Passed. StudyHours and Attendance are input features. Passed is the target classification label, where 0 represents Fail and 1 represents Pass.

```
StudyHours,Attendance,Passed
1,50,0
2,55,0
2,60,0
3,65,0
4,70,1
5,72,1
5,75,1
6,78,1
7,80,1
8,85,1
9,90,1
10,95,1
```

ML Task Implementation

For this task, I used Pandas to load the classification dataset and selected StudyHours and Attendance as input features. The data was divided into training and testing sets. A Logistic Regression model was trained on the training data, then used to predict the test labels. Finally, the accuracy score was calculated to evaluate the classification results.

ML Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

print("=====")
print("  ML WEEK 4 - CLASSIFICATION")
print("=====")

# Load classification dataset
df = pd.read_csv("student_classification.csv")

print("\n===== DATASET =====")
```

```

print(df)

# Features and target
X = df[["StudyHours", "Attendance"]]
y = df["Passed"]

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Create classification model
model = LogisticRegression()

# Train model
model.fit(X_train, y_train)

print("\n===== MODEL TRAINING =====")
print("Logistic Regression model trained successfully!")

# Predict test data
y_pred = model.predict(X_test)

print("\n===== PREDICTIONS =====")
print("Actual values: ", list(y_test))
print("Predicted values:", list(y_pred))

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\n===== ACCURACY =====")
print(f"Accuracy Score: {accuracy * 100:.2f}%")

# Predict new students
new_students = pd.DataFrame({
    "StudyHours": [3, 6, 9],
    "Attendance": [60, 80, 90]
})

new_predictions = model.predict(new_students)

print("\n===== NEW PREDICTIONS =====")

for i, prediction in enumerate(new_predictions):
    result = "Pass" if prediction == 1 else "Fail"

    print(
        f"Student {i + 1}: "
        f"Study Hours = {new_students.iloc[i]['StudyHours']}, "
        f"Attendance = {new_students.iloc[i]['Attendance']}% "
        f"-> Prediction: {result}"
    )

print("\n=====")
print("    WORKING CLASSIFICATION MODEL")
print("=====")

```

ML Code Screenshot

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import accuracy_score
5
6 print("=====")
7 print("    ML WEEK 4 - CLASSIFICATION")
8 print("=====")
9
10 # Load classification dataset
11 df = pd.read_csv("student_classification.csv")
12
13 print("\n===== DATASET =====")
14 print(df)
15
16 # Features and target
17 X = df[["StudyHours", "Attendance"]]
18 y = df["Passed"]
19
20 # Split dataset into training and testing data
21 X_train, X_test, y_train, y_test = train_test_split(
22     X,
23     y,
24     test_size=0.25,
25     random_state=42,
26     stratify=y
27 )
28
29 # Create classification model
30 model = LogisticRegression()
31
32 # Train model
33 model.fit(X_train, y_train)

```

```

34
35 print("\n===== MODEL TRAINING =====")
36 print("Logistic Regression model trained successfully!")
37
38 # Predict test data
39 y_pred = model.predict(X_test)
40
41 print("\n===== PREDICTIONS =====")
42 print("Actual values: ", list(y_test))
43 print("Predicted values:", list(y_pred))
44
45 # Calculate accuracy
46 accuracy = accuracy_score(y_test, y_pred)
47
48 print("\n===== ACCURACY =====")
49 print(f"Accuracy Score: {accuracy * 100:.2f}%")
50
51 # Predict new students
52 new_students = pd.DataFrame({
53     "StudyHours": [3, 6, 9],
54     "Attendance": [60, 80, 90]
55 })
56
57 new_predictions = model.predict(new_students)
58
59 print("\n===== NEW PREDICTIONS =====")
60
61 for i, prediction in enumerate(new_predictions):
62     result = "Pass" if prediction == 1 else "Fail"
63

```

```

59 print("\n===== NEW PREDICTIONS =====")
60
61 for i, prediction in enumerate(new_predictions):
62     result = "Pass" if prediction == 1 else "Fail"
63
64     print(
65         f"Student {i + 1}: "
66         f"Study Hours = {new_students.iloc[i]['StudyHours']}, "
67         f"Attendance = {new_students.iloc[i]['Attendance']}% "
68         f"-> Prediction: {result}"
69     )
70
71 print("\n=====")
72 print("    WORKING CLASSIFICATION MODEL")
73 print("=====")

```

ML Week 4 — Classification Model Code Screenshot

ML Output Screenshot

```

... =====
      ML WEEK 4 - CLASSIFICATION
=====

===== DATASET =====
      StudyHours  Attendance  Passed
0              1           50       0
1              2           55       0
2              2           60       0
3              3           65       0
4              4           70       1
5              5           72       1
6              5           75       1
7              6           78       1
8              7           80       1
9              8           85       1
10             9           90       1
11            10           95       1

===== MODEL TRAINING =====
Logistic Regression model trained successfully!

===== PREDICTIONS =====
Actual values:  [0, 1, 1]
Predicted values: [np.int64(0), np.int64(1), np.int64(1)]

===== ACCURACY =====
Accuracy Score: 100.00%

===== NEW PREDICTIONS =====
Student 1: Study Hours = 3, Attendance = 60% -> Prediction: Fail
Student 2: Study Hours = 6, Attendance = 80% -> Prediction: Pass
Student 3: Study Hours = 9, Attendance = 90% -> Prediction: Pass

```

ML Week 4 — Classification Model Output Screenshot

ML Workflow Explanation

- Load the classification dataset using Pandas.
- Select StudyHours and Attendance as input features.
- Use Passed as the target classification label.
- Split the dataset into training and testing data.
- Create and train the Logistic Regression model.
- Predict the labels of the test data.
- Calculate the accuracy score.
- Use the trained model to classify new student examples.

ML Result

The final output should demonstrate that the Logistic Regression model was trained successfully, generated predictions, and produced an accuracy score. The actual Google Colab output should be captured and inserted into the screenshot area above.

4. Overall Conclusion

Week 4 provided practical experience with two important Machine Learning concepts. In the AI task, I implemented a basic neural network for prediction. In the ML task, I implemented a Logistic Regression classification model and evaluated its predictions using an accuracy score. Together, these tasks demonstrate the basic workflow of creating, training, and evaluating Machine Learning models.

5. Submission Information

Program	DevSphere Internship Program
Intern	Saud Masood
Domain	Artificial Intelligence & Machine Learning
Week	04
Report Date	23 September 2026